CAMP ガイド
人と AI が一緒に決める場の、通信の約束

Collaborative Agent Mediation Protocol ／ `@camp-protocol/kit` 0.1.0 ／ 2026-11-09 時点（なりすまし防止の修正を含む） ／ チーム VICTORY

> **一言でいうと**：何人もの人が同時に操作し、その間に遅い AI の結果が届く場で、**全員の画面を同じ順番・同じ状態にそろえる**ための約束（プロトコル）と、その部品（npm パッケージ）。
> オーディションか会議かといったアプリの中身は知らない。HTTP が「買い物かご」を知らないのと同じで、**どんな命令があるか**はアプリが渡す。

1 どこにいるのか

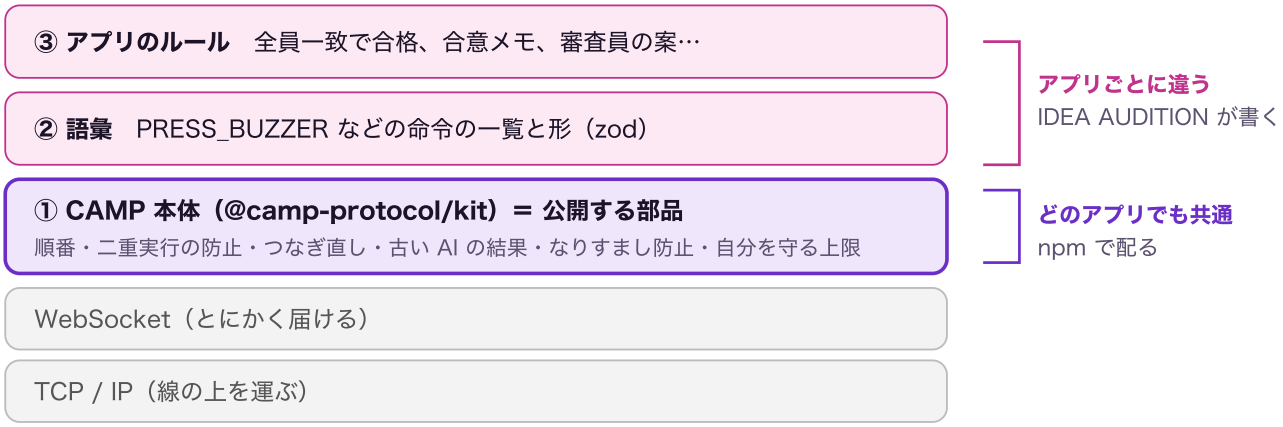


図1　CAMP は WebSocket の上、アプリの下にある共通の層

WebSocket だけだと、アプリが自分で解かないといけないこと

困りごと	例（IDEA AUDITION）	CAMP の答え
同時に操作されたら、どれを先に確定するか	4人がほぼ同時にブザーを押す	部屋ごとに1件ずつ処理し、確定に通し番号 `seq`
返事が来ず送り直したら、二度実行されないか	通信が切れて「合格」が2回確定する	`commandId` で前の結果を返すだけ
切れてつなぎ直したら、取りこぼしはどうなるか	トンネルで切れた人の画面だけ古い	welcome で状態の全体を渡す
遅れて届いた AI の結果を、そのまま使ってよいか	やり直したのに古い案が出る	epoch が違えば捨てる
他人のふりをされないか 今回	ホストの ID を名乗って優勝を決める	本人にだけ渡す合言葉

2 3つの層の分担（IDEA AUDITION の場合）

層	場所	中身
① CAMP 本体	`packages/camp`	封筒の形、WebSocket の受け口、版の確認、形の検査のしくみ、二重実行の防止（ResultCache）、順番（SeqGate）、クライアント（つなぎ直し・送り直し）、汎用の部屋（CampRoom：epoch と task）、合言葉
② 語彙	`packages/protocol`	`defineVocabulary` に渡す命令（PRESS_BUZZER・CAST_VOTE…）、イベントの名前、断る理由、snapshot の形
③ アプリのルール	`apps/server`（SessionActor・AuditionSession）	全員一致の判定、持ち時間、決勝、合意メモ、AI への依頼と古い結果の扱い、保存（Firestore）

新しい機能（たとえば「審査員も案を出す」）は、②に命令を1つ、③に処理を足すだけで、①は変わらない。①は命令の中身を見ずに、「誰の・どの番号の命令か」「順番」「二重」「古さ」だけを扱う。

> **正確に言うために**：IDEA AUDITION のサーバーは、①の汎用の部屋（CampRoom）は使わず、同じ考え方（1件ずつ処理・epoch・合言葉）を SessionActor で自分で実装している。①から使っているのは、封筒・形の検査・WebSocket の受け口・ResultCache・クライアント・合言葉の部品。「CAMP の仕様に沿ったアプリ」であり、すべての保証が npm の部品の中にあるわけではない。

3 メッセージは7種類

種類	向き	返事・保存	使いどころ
`hello`	クライアント→	—	つないだ直後に1回。部屋 ID、前の参加者 ID と**合言葉**、最後に受け取った seq
`welcome`	→クライアント	—	参加者 ID（満員なら null＝観覧）、**合言葉（本人にだけ）**、状態の全体
`command`	クライアント→	result が必ず返る	状態を変える頼みごと。`commandId` 付き
`result`	→本人	—	成功か、断った理由（UPPER_FULL・NOT_HOST…）
`event`	→全員	通し番号 seq	確定した変化と、その時点の状態の全体
`ephemeral`	どちらも	保存しない	カーソル、マイクのオンオフ、通話をつなぐ合図。送り手はサーバーが付け直す
`signal`	クライアント→	返事なし・間引く	「うーん」など言葉にならない反応。1人1秒の件数まで

このほか、直せる情報を返す `protocol_error`（版違い・壊れた形・hello 前・送りすぎ・部屋が無い）がある。

4 1つの接続の流れ

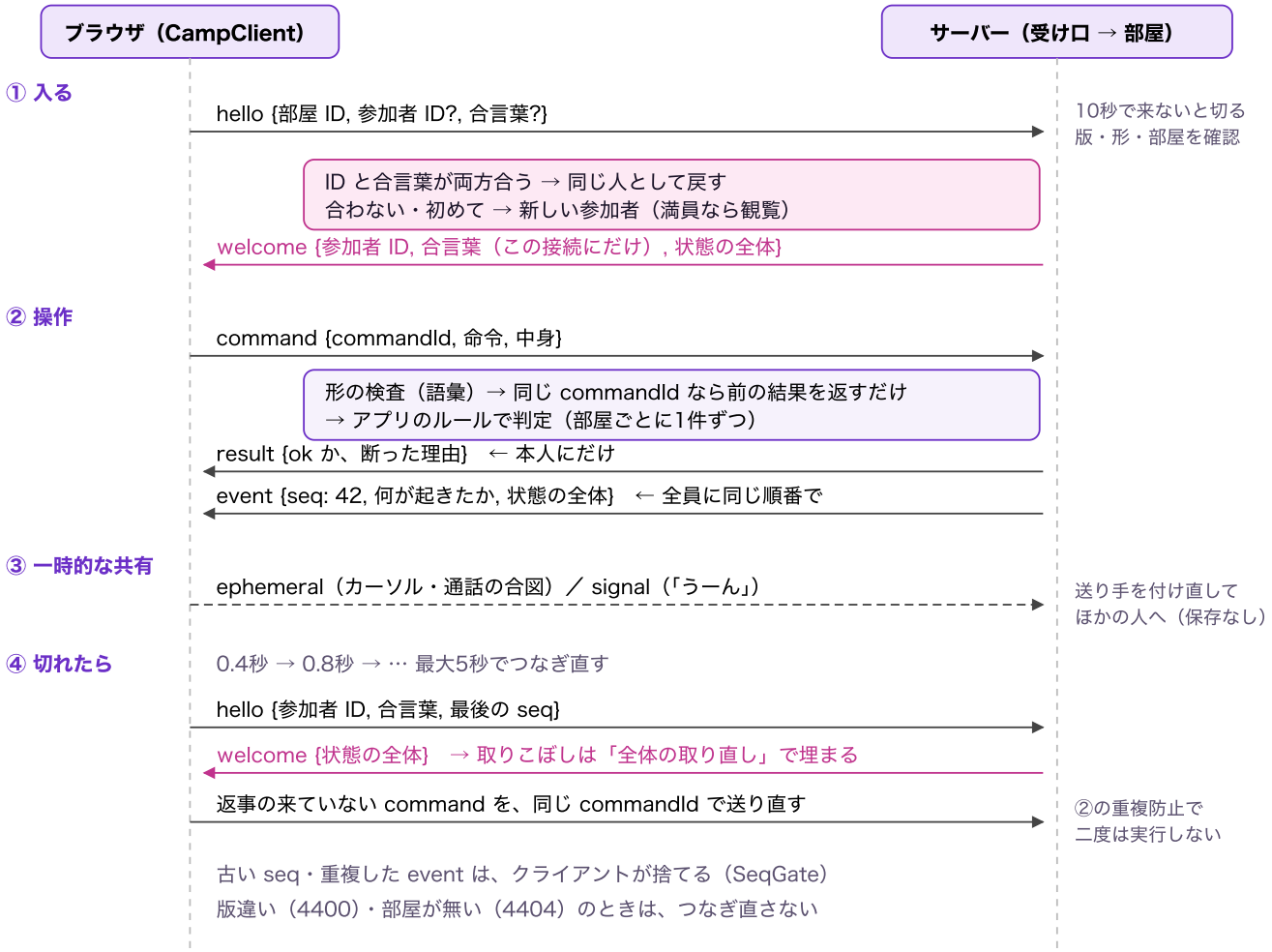
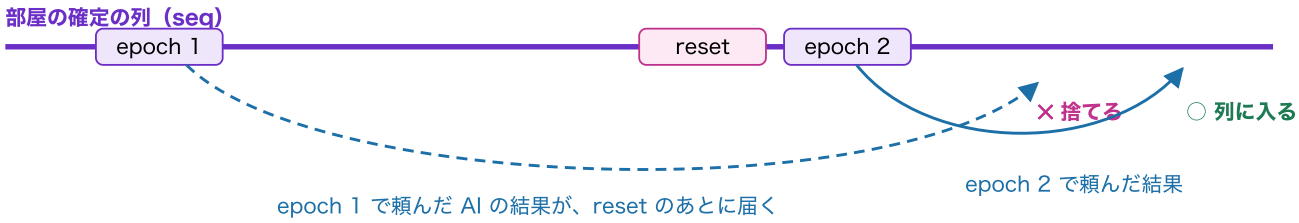


図2　hello から切断・復帰まで

5 遅い AI の結果：epoch



AI の結果は人の操作と同じ列に並び、同じ seq で全員に届く。
反映の途中で例外が出ても、記録するだけでサーバーは止まらない**（今回の修正）**。
「合意が変わったら古い前提の案を捨てる」は、IDEA AUDITION（③）が合意の版で同じ考え方をしている。

図3　頼んだときの epoch と今が違えば、結果を捨てる

人と AI の扱いの違い

	人（参加者）	AI
入り方	ブラウザから command を送る。commandId を持ち、result が返る	参加者ではない。サーバーが頼み、あとから結果が届く
二重の防止	同じ commandId は二度実行しない	頼んだときの epoch（や合意の版）を覚えておく
古くなったら	捨てない。ルールに合わなければ理由つきで断る	黙って捨てる
役割	決める（ブザー・投票・賛否）	提案する（案・改良・回答）
同じところ	確定するときは同じ列に並び、同じ seq で全員に届く	

6 なりすまし防止：合言葉 今回の修正

前の問題
参加者 ID（`p_1`・審査員 ID）は、状態の中で全員に配られる。hello にその ID を付けるだけで、その人として戻れた。部屋のリンクを知っている人なら、ホストを含む誰にでもなりすませた。

今の作り
入ったとき、本人の接続にだけ合言葉（24バイトの乱数）を渡す。状態やイベントには入れない。つなぎ直しは ID と合言葉が**両方**合うときだけ。合わなければ新しい参加者として入る。比べるときは、時間差で中身が漏れない方法（timingSafeEqual）を使う。

ブラウザは ID と合言葉をこのタブの sessionStorage に覚える。だから再読み込みしても同じ人に戻れ、ほかの人は席を奪えない。v1 の中で任意の項目を足しただけなので、版は上げていない（仕様書 §10 の「任意の項目は足してよい」）。

合言葉は具体的にどういうものか

人が覚えて打ち込むパスワードではない。サーバーが作り、ブラウザが裏で覚えておく**ランダムな文字列**（コードでは `resumeToken`）。

項目	中身
形	暗号用の乱数24バイトを、URL に使える文字にした32文字。例（形のイメージ）：`q3Vf9xL0aK2-ZbT7mWc1Rr8uYdE5sN4h`
作り方	`randomBytes(24).toString('base64url')`（`server/resume-token.ts`）。当てずっぽうで当たる確率は実質ゼロ
だれが知っているか	サーバーと本人のブラウザだけ。状態（snapshot）にもイベントにも入れない
どこに残るか	そのタブの sessionStorage。同じタブで再読み込みしても戻れ、新しいタブで開くと別の人として入る
いつ消えるか	席が消えるとき（満員で、切断した人の席を空けたときなど）

順番	やりとり
① はじめて入る	サーバーが参加者 ID（例 `p_1`）と合言葉を作り、welcome でその人の接続にだけ送る
② 覚える	ブラウザが ID と合言葉の組を sessionStorage に入れる
③ 切れて戻る	hello に「ID ＋ 合言葉」を付けて送る
④ 照らし合わせる	一致すれば同じ人として席に戻す。合わなければ別の新しい参加者として入れる

参加者 ID
状態に入り、全員の画面に届く。だれが投票したか・だれがホストかを出すのに必要。

合言葉
本人の welcome にしか乗らない。ほかの人は知りようがない。ID だけ名乗っても戻れない。

7 自分を守る仕組みと、アプリに任せること

仕組み	値・中身
hello の待ち時間	10秒（来なければ 4408 で切る）
1通の大きさ	64KB まで（アプリの設定で変えられる）
1接続の通数	1秒120通まで（超えたら 4429 で切る）
生存確認	25秒ごとに ping。応えなければ切る
部屋の数 今回	手軽な起動のしかた（`createCampServer`）では最大100
部屋 ID	`kqd-rmzw-tbe` の形。26の10乗通り、暗号用の乱数
送り手	クライアントの名乗りは信じず、接続から決める
形の検査	すべてのメッセージを zod で検査。知らない命令は UNKNOWN_TYPE、形違いは INVALID_PAYLOAD
知らないパスへの接続	何も返さず切らずに HTTP 404 で断る（前段に「壊れた応答」と見なされてサーバーごと入れ替えられないように）

CAMP が守ること（1つの部屋・1つのサーバーの中で）

- 全員が同じ変化を同じ順番（seq）で受け取る
- 1つの操作は二度実行されない（部屋ごとに直近500件を覚える）
- reset 前に始めた AI の結果は状態に入らない
- つなぎ直せば最新の状態にそろう
- 他人の席は奪えない

アプリに任せること

- ルール（誰が何をしてよいか）
- ログイン（誰なのか）。WebSocket の手前か、部屋の登録で確かめる
- 部屋づくりの回数の制限（1人あたり）
- 保存（汎用の部屋はメモリだけ）
- 1つの部屋を複数のサーバーで動かすこと（1つの部屋は1つのサーバーに固定する）
- 規模：イベントのたびに状態の全体を配るので、少人数の部屋向け

8 部品とファイル（ソース 約1,240行）

ファイル	行	役目
core/（どこでも動く。zod だけに依存）		
`version.ts`	17	版 "1"、終了コード（4400・4404・4408・4429）
`envelope.ts`	96	封筒の形（hello・command・event・welcome・result・protocol_error）
`vocabulary.ts`	196	`defineVocabulary`：アプリの命令一覧から、検査用のスキーマと型を作る
`idempotency.ts`	26	ResultCache：送り手と commandId ごとに結果を覚える（古いものから捨てる）
`sequence.ts`	23	SeqGate：古い・重複した seq を捨てる
client/（ブラウザでも Node でも動く）		
`camp-client.ts`	250	つなぎ直し、送り直し、順番、合言葉の保持、版違い・部屋なしでは止まる
server/（Node。ws に依存）		
`transport.ts`	192	WebSocket の受け口：hello の待ち時間、版、形、送りすぎ、生存確認、部屋への振り分け
`room.ts`	267	汎用の部屋 CampRoom：1件ずつ処理、seq と配信、席とホスト、観覧、task と epoch、合言葉
`server.ts`	112	createCampServer：http・/camp・部屋の管理をまとめて立てる（部屋は最大 maxRooms）
`resume-token.ts` 新	18	合言葉を作る・比べる（自前の部屋でも使える）
`room-id.ts`	17	推測されにくい部屋 ID を作る・形を確かめる
`upgrade.ts`	11	知らないパスへの接続を HTTP 404 で断る

9 公開するファイルのすべて

公開は2か所。公開リポジトリ は `github.com/camp-protocol/camp`（`packages/camp` をそのまま切り出す、34ファイル）。npm は `@camp-protocol/kit@0.1.0`（80ファイル、圧縮して約53KB。`package.json` の `files` にあるものだけ）。

ファイル	どこに出るか	中身
`package.json`	両方	名前・版・依存（ws・zod）・入口・著者・ライセンス
`LICENSE`	両方	MIT
`README.md`・`README.ja.md`	両方	使い方（英語・日本語）
`SPEC.md`	両方	仕様書（英語）。準拠の条件10項目
`src/`（15ファイル）	両方	上の8章のソース（TypeScript）
`dist/`（60ファイル）	npm だけ	src を JavaScript にしたもの・型定義（.d.ts）・対応表（.map）。公開のときに自動で作る（リポジトリには入れない）
`test/boundary.test.ts`・`test/quick-vote.test.ts`	リポジトリだけ	テスト19件（境界5件＋Quick Vote で準拠の条件など14件）
`examples/quick-vote/`（4ファイル）	リポジトリだけ	IDEA AUDITION と関係のない最小の例（みんなで投票）
`assets/`（3ファイル）	リポジトリだけ	ロゴ（正方形のアイコン512px・横長のロゴ）
`scripts/prepack.mjs`	リポジトリだけ	npm ではなく pnpm で公開することを強制（npm だと入口が src を指してしまう）
`scripts/check-pack.mjs`	リポジトリだけ	固めたものを空のプロジェクトに入れて、3つの入口が読み込めるか確かめる
`.github/workflows/ci.yml`	リポジトリだけ	Node 20・22・24 で、型・テスト・check:pack を回す
`tsconfig.json`・`tsconfig.build.json`	リポジトリだけ	型チェックと、dist を作る設定

npm パッケージの中身（80ファイル）

```
package.json
LICENSE
README.md
README.ja.md
SPEC.md
src/core/envelope.ts
src/core/idempotency.ts
src/core/index.ts
src/core/sequence.ts
src/core/version.ts
src/core/vocabulary.ts
src/client/camp-client.ts
src/client/index.ts
src/server/index.ts
src/server/resume-token.ts
src/server/room-id.ts
src/server/room.ts
src/server/server.ts
src/server/transport.ts
src/server/upgrade.ts
dist/core/ envelope・idempotency・index・sequence・version・vocabulary（各 .js / .d.ts / .js.map / .d.ts.map）
dist/client/ camp-client・index（同上）
dist/server/ index・resume-token・room-id・room・server・transport・upgrade（同上）
```

> **公開しないもの**：IDEA AUDITION のコード（apps・packages/protocol）、.env、GCP のプロジェクト名やサービスアカウント。切り出しスクリプト（`scripts/camp-split.sh`）が、`idea-audition-run`・`GOOGLE_CLOUD_PROJECT`・`.iam.gserviceaccount.com` などの文字が入っていたら止まる。境界テストが、CAMP が IDEA AUDITION を import していないことを確かめる。

10 公開の手順と、その前に決めること

2026-11-10 に公開済み：公開リポジトリ camp-protocol/camp（英語1コミット・CI 成功）と npm の `@camp-protocol/kit@0.1.0`。アプリは npm の版を使っている。

1. **済み**：個人のメールは出さない。`package.json` の著者欄は名前と GitHub の URL だけ。公開用のコミットの作者は GitHub の noreply アドレス。npm のアカウントのメールは、公開のデータ（maintainers）に入る
2. **決める**：npm のアカウントの2段階認証。GitHub Actions から出所の証明（provenance）付きで公開するか
3. 公開前の確認（秘密の情報・アプリの言葉が入っていないか、境界テスト）
4. `scripts/camp-split.sh` で英語の1コミットを作り（noreply の作者・Gmail の文字が入っていたら止まる）、公開リポジトリを置き換える（今の日本語の5コミットと古い Actions の記録を消す）
5. 公開リポジトリの CI（Node 20・22・24）が通る
6. `pnpm publish --access public` → `npm view @camp-protocol/kit` で見える
7. アプリの package.json を `workspace:*` から npm の版に切り替え、本番で確かめる

記録：issue #23 のコメント。今のところ、残りの issue（#1〜#42）で CAMP 本体の作りを変える必要は見込んでいない（実測のベンチマークを足すのは、公開後でよい）。

11 用語

言葉	意味
部屋（room）	1つの共有の場。IDEA AUDITION では1回のオーディション
参加者（participant）	操作できる人。IDEA AUDITION では審査員（4人まで）。超えた人は観覧
commandId	命令ごとの番号。同じ番号は二度実行しない
seq	確定した変化の通し番号。全員がこの順番で受け取る
snapshot	その時点の状態の全体。event と welcome に入る
epoch	部屋の「やり直し」の回数。頼んだときと違えば、AI の結果を捨てる
合言葉（resumeToken）	本人にだけ渡す、つなぎ直し用の秘密。参加者 ID と組で使う
語彙（vocabulary）	アプリの命令・イベント・断る理由・状態の形の一覧